

Abstract

This report describes the complete set of lossless transformations used in the PAQJP 8.8 compression system. It covers all 256 single-byte-stream transformations (numbered 1–256) and the 2704 ordered transformation pairs built from the first 52 of those transformations. Every transform is mathematically invertible, guaranteeing that no information is lost during forward and reverse operations. The document includes all relevant formulas, derivations of invertibility, and the exact combinatorial enumeration of the 2704 pair sequences. The total number of available transform paths (including the raw “no-transform” option) is $256 + 2704 + 1 = 2961$.

1. Introduction

Lossless compression often benefits from a preprocessing step that rearranges or decorrelates the input data before passing it to a statistical backend. PAQJP 8.8 implements a very large set of such preprocessing transforms, all of which are mathematically invertible byte-by-byte operations. The system offers:

- 256 single transforms ($T_1, \dots, T_{256}$).
- 2704 ordered transform pairs formed as $T_{t_2} \circ T_{t_1}$ for $t_1, t_2 \in \{1, 2, \dots, 52\}$.
- The raw (no-transform) path.

During compression, every one of these 2961 pipelines is tried, and the one yielding the smallest backend-compressed payload is selected. The selected pipeline is signalled in a variable-length header. Decompression simply reads the header, applies the inverse transforms, and obtains the original data exactly.

2. Transform Categories and Mathematical Formulations

Each transform T maps a byte string $\mathbf{d} = (d_0, d_1, \dots, d_{n-1})$, $d_i \in \{0, 1, \dots, 255\}$, to another byte string $\mathbf{d}'$ of possibly the same length (most transforms) or slightly longer (when metadata is embedded). The inverse T^{-1} recovers $\mathbf{d}$ from $\mathbf{d}'$.

We group the 256 transforms according to their mathematical nature. The formulas below are exact and define the forward transformation; the inverse is obtained by reversing the order of operations or applying the algebraic inverse of each step.

2.1 Run-Length Encoding (Transform 1, code name transform_00)

This is the most complex single transform. It searches for a series of additive shifts that maximise the sum of squares of run lengths, then encodes the shifted data with a custom RLE bitstream.

Forward:

1. For at most 10 iterations, find an additive shift $s \in [0, 255]$ that maximises $\sum r_i^2$ where r_i are the run lengths of equal bytes after adding s to all bytes modulo 256.
2. Apply the best shift, record it. Repeat the process on the shifted result. Keep the sequence of shifts that gives the smallest final RLE-encoded length.
3. Encode the number m of shifts applied (a byte) followed by the list of shift values, then the RLE bitstream.

RLE bitstream format:

The shifted data is encoded with variable-length codes:

- Single byte: 00 + 8-bit value.
- Short run (2–5): 01 + 2-bit run-length-minus-2 + 8-bit value.
- Medium run (6–12): 10 + 3-bit run-length-minus-6 + 8-bit value.
- Long run (13–268): 1111 + 8-bit (run-length-minus-13) + 8-bit value.

The inverse first decodes the RLE stream to recover the shifted data, then subtracts the recorded shifts in reverse order.

Losslessness: The RLE decoder is deterministic and reproduces exactly the shifted sequence. Subtracting the shifts modulo 256 is invertible. Therefore $T_{-1} \circ T_1 = \text{id}$.

2.2 XOR with Fixed or Computed Byte Masks (Transforms 2, 10, 16, 22, 23–255 many dynamic ones)

A large family of transforms XOR each byte with a constant or a position-dependent value.

General formula:

$$d'_i = d_i \oplus k_i \pmod{256}$$

where k_i is either a constant or derived from the data length, checksums, prime numbers, digit expansions of constants, Fibonacci numbers, or pseudo-random sequences seeded by length.

Examples (with exact formulas):

- Transform 2 (transform_01): Iterates over all primes <256, computes $\text{xor_val} = \text{ceil}(\text{prime} * 4096 / 28672)$ (but for prime=2 it uses prime itself), then XORs every 3-rd byte with that value, repeating 100 passes. This is a symmetric XOR because $(a \oplus k) \oplus k = a$.

- Transform 11 (transform_10): Counts occurrences of the two-byte pattern "X1" (bytes 0x58, 0x31), computes $n = ((\text{count} \cdot 2 + 1) // 3) \cdot 3 \bmod 256$, and XORs all bytes with n . The inverse is identical: XOR again with n .

- Transform 18 (transform_17): Uses the lossless k derived from π for 7 digits:

$$k = \text{round}(\lceil (\pi - 3) \times 16777216 \rceil)$$

The bit length of k (23 or 25 bits) determines the mask length; the mask bytes are the binary expansion of k . XOR all bytes with the repeating mask. Inverse identical.

- Transform 21 (transform_20): Mask from the digits of $5e$:

$$5e = 5 \times \exp(1) \approx 13.59140914 \dots$$

Digits are paired to form mask bytes; all bytes XORed with the cyclic mask. Inverse identical.

- Transforms 23–255: Use a seed derived from the transform number and data length: $\text{seed} = \text{get_seed}(\text{transform_num} \% 126, \text{data_length})$. All bytes XORed with that seed. Inverse identical.

Invertibility: XOR with any deterministic mask is its own inverse because $(d \oplus k) \oplus k = d$. No additional metadata is required for the mask as it can be recomputed from the same context (data length, transform number). Transform 2 is also self-inverse because XOR is applied 100 times, which is an even number of passes? Actually it's 100 iterations of a loop, but each iteration re-XORs the same bytes with the same `xor_val`, so the net effect is toggling the value 100 times: if 100 is even, the result is identity; if odd, it's a single XOR. The code shows for `_` in `range(r)`: where `r=100` (even), so it is identity! But the transform is still lossless. This demonstrates that even redundant operations preserve invertibility.

2.3 Additive/Subtractive Shifts (Transforms 5, 22)

- Transform 5 (transform_04): For 100 iterations, subtracts $(i \bmod 256)$ from byte at position i :

$$d_{i^{(k+1)}} = (d_{i^{(k)}} - (i \bmod 256)) \bmod 256$$

Inverse adds $(i \bmod 256)$ 100 times.

- Transform 22 (transform_21): Adds a constant 255 to every byte modulo 256. Inverse subtracts 255 modulo 256, which is the same as adding 1.

2.4 Rotation (Cyclic Bit Shifts) (Transforms 4, 6)

- Transform 4 (transform_03): Applies a rotation by rotation bits to every 5-th byte starting at index 2:

$$\text{rot}_r(x) = \big((x \ll r) \vee (x \gg (8-r))\big) \bmod 256$$

The rotation amount r is computed from data length and checksum modulo 8. Inverse uses $8 - r$ rotation.

- Transform 6 (transform_05): Rotates every byte left by a fixed 3 bits. Inverse rotates right by 3 bits.

2.5 Substitution Cipher (Transform 8)

- Transform 8 (transform_06): Generates a permutation of $[0,255]$ using a fixed seed (42), then substitutes each byte. Inverse applies the inverse permutation. This is perfectly invertible.

2.6 Prime-Based and Seed-Based XOR (Transforms 9, 10, 13, 14)

- Transform 9 (transform_08): XORs each byte with the nearest prime to $(\text{length} \bmod 256)$, then for 100 passes XORs with a rotated PI_DIGITS mask. Self-inverse.
- Transform 10 (transform_09): Similar but also includes a seed value.
- Transform 14 (transform_13): Computes repeats from length and checksum, finds repeats successive nearest primes, XORs all bytes with the last prime. Stores repeats-1 as a header byte. Inverse recalculates the same prime chain and XORs with the last prime.

2.7 Padding and Checksum (Transform 15)

- Transform 15 (transform_14): Appends a single byte containing the arithmetic sum of all input bytes modulo 256. Inverse simply truncates the last byte. This is invertible because the original data is a prefix of the transformed data.

2.8 Pattern-Based Additive Modifications (Transforms 3, 16)

- Transform 3 (transform_02): Computes a pattern of 4 bytes from a seed (based on length+checksum), then XORs every 4-th byte (starting at index 1) with that pattern. Stores the pattern index. Inverse recovers the pattern and XORs again.
- Transform 16 (transform_15): Similar but uses addition modulo 256 instead of XOR, and a 3-byte pattern on every 3-rd byte (starting at 0). Inverse subtracts.

2.9 Fibonacci and Position-Dependent Keys (Transforms 12, 13)

- Transform 12 (transform_11): Computes a key for byte i as

$$k_i = (F_{(i+\text{len})} \bmod |F|) \oplus ((i \cdot 13 + \text{len} \cdot 17) \bmod 256) \bmod 256$$

where F is the Fibonacci sequence (mod 256). XORs each byte with k_i . Inverse identical.

- Transform 13 (transform_12): XORs with the Fibonacci sequence directly (mod 256). Self-inverse.

2.10 PI-Based Masks (Transforms 17–21)

These transforms (17–21) generate variable-length masks from the decimal digits of mathematical constants:

- Transform 17 (transform_16): Simple XOR with constant $(\text{len} \cdot 7 + 13) \% 256$.

- Transform 18 (PI-K mask), described earlier.
- Transform 19 (transform_18): Mask from Basel digits $\pi^2/6$.
- Transform 20 (transform_19): Mask from $1/e$.
- Transform 21 (transform_20): Mask from $5e$.

All are XOR masks; inverses are identical.

2.11 Identity Transform (Transform 256)

$T_{256}(\mathbf{d}) = \mathbf{d}$. Its inverse is itself.

3. Transform Pairs: 2704 Ordered Combinations

The compression system also considers applying two transforms in sequence. Let T_a and T_b be two single transforms. The combined transform is $T_b \circ T_a$, defined as:

$$(T_b \circ T_a)(\mathbf{d}) = T_b(T_a(\mathbf{d}))$$

The inverse is $T_a^{-1} \circ T_b^{-1}$.

3.1 Selection of the Base Transform Set

Only the first 52 single transforms (numbered 1 through 52) are used as the building blocks for pairs. This choice balances the combinatorial explosion ($52 \times 52 = 2704$) with the need to keep compression time feasible.

3.2 Pair Enumeration

All ordered pairs (t_1, t_2) with $t_1, t_2 \in \{1, 2, \dots, 52\}$ are generated. The pair is indexed from 0 to 2703 using the formula:

$$\text{idx} = (t_1 - 1) \times 52 + (t_2 - 1)$$

Thus, the pair (1,1) has index 0, (1,2) index 1, ..., (52,52) index 2703. The total number of pairs is:

$$52 \times 52 = 2704$$

3.3 Pair Application and Inversion

For compression, given a pair (t_1, t_2) , the input data is transformed as:

$$\mathbf{d}' = T_{t_2} \big(T_{t_1}(\mathbf{d}) \big)$$

For decompression, the inverse pair $(t_1, t_2)^{-1} = (t_2^{-1}, t_1^{-1})$ is applied to the decompressed backend data:

$$\mathbf{d} = T_{t_1}^{-1} \big(T_{t_2}^{-1}(\mathbf{d}') \big)$$

Because each single transform is invertible, the composite is invertible.

3.4 Losslessness Guarantee

Since every single transform T_i satisfies $T_i^{-1}(T_i(x)) = x$, we have:

$$T_{\{t_1\}^{-1}}(T_{\{t_2\}^{-1}}(T_{\{t_2\}}(T_{\{t_1\}}(x)))) = T_{\{t_1\}^{-1}}(T_{\{t_1\}}(x)) = x$$

Hence all 2704 pair transforms are mathematically lossless.

4. Header Format and Transform Selection Signalling

The compressed bitstream begins with a variable-length header that indicates which transform pipeline was used:

- Single transform T_k ($1 \leq k \leq 252$): header byte $k-1$.
- Extended single T_k ($253 \leq k \leq 256$): header byte 254 followed by one byte $k-253$.
- Raw (no transform): header byte 252.
- Transform pair: header byte 253 followed by two bytes encoding the pair index in big-endian (0..2703).

During decompression, the header is parsed to obtain:

- For singles: a tuple containing one transform number.
- For pairs: a tuple (t_1, t_2) .
- For raw: an empty tuple.

The backend-compressed payload follows the header. The decompressor first decodes the backend, then applies the inverse transforms in the correct order.

5. Example Calculations

Single Transform Example – Transform 22 (add 255):

Let $\mathbf{d} = [0, 128, 255]$.

· Forward: $d_i' = (d_i + 255) \bmod 256$.

```
\begin{aligned}
0 + 255 &= 255 \rightarrow 255 \\
128 + 255 &= 383 \bmod 256 = 127 \\
255 + 255 &= 510 \bmod 256 = 254
\end{aligned}
```

Transformed: $[255, 127, 254]$.

· Inverse: $d_i = (d_i' - 255) \bmod 256 = (d_i' + 1) \bmod 256$.

```
\begin{aligned}
255 + 1 &= 256 \bmod 256 = 0 \\
127 + 1 &= 128 \\
254 + 1 &= 255
\end{aligned}
```

Original recovered.

Pair Transform Example – Transforms 5 and 6 (additive-shift then rotate):

Take T5 (subtract position index 100 times) and T6 (rotate left 3 bits).

· Suppose input $[0,0]$. For simplicity, compute one byte:

· T5: after 100 iterations of subtracting 0 (since $i=0$) the byte remains 0.

· T6: rotate left 3 bits of 0 gives 0.

· Inverse: apply $T6^{-1}$ (rotate right 3 bits) then $T5^{-1}$ (add index 100 times). Result is 0.

To show a non-trivial case, take input [1].

- T5: subtract (0 mod 256) from byte at i=0 $\rightarrow 1 - 0 = 1$.
- T6: rotate left 3 bits of 1: binary 00000001 rotated left 3 gives 00001000 = 8.
- Inverse T6⁻¹: rotate right 3 bits of 8: 00001000 $\rightarrow$ 00000001 = 1.
- Inverse T5⁻¹: add 0 $\rightarrow$ 1. Original recovered.

Pair Index Calculation:

For pair (5, 6): $\text{idx} = (5-1) \times 52 + (6-1) = 4 \times 52 + 5 = 208 + 5 = 213$.

Header: 253 followed by 0x00, 0xD5 (213 in big-endian).

6. Summary of Transform Counts

Category Count

Single transforms (1–256) 256

Transform pairs (1..52 $\times$ 1..52) $52^2 = 2704$

Raw path 1

Total available pipelines 2961

All 2961 pipelines are mathematically lossless. The compressor selects the one that minimises the size of the backend-compressed output. The self-test in the source code verifies every single transform on all 256 byte values and every pair on all 256 byte values, confirming 100% bit-accurate invertibility.

7. Conclusion

The PAQJP 8.8 transform system is a carefully designed ensemble of invertible byte-stream transformations that provide a rich set of decorrelation possibilities. The mathematical

foundation relies on simple, reversible operations such as XOR, modular addition/subtraction, rotation, substitution, and run-length encoding with embedded metadata. The combinatorial extension to 2704 ordered pairs multiplies the number of effective filters, increasing the chance that some transform exposes statistical redundancies that the backend compressor (zstd or PAQ) can exploit. Every transform and every pair is accompanied by an exact inverse, guaranteeing lossless reconstruction under all circumstances. The header encoding of the chosen pipeline is efficient, requiring only 1–3 bytes for any of the 2961 possibilities. This design achieves a high degree of compression-oriented data shaping while preserving mathematical exactness.